Universitatea Tehnică de Construcții București
Facultatea de Hidrotehnica

Proiect la PCLP II

Profesor Coordonator
Fudulu Cosmin
Olteanu Gabriela

Student
Tudor Constantin Mihai
Ilie Robert Stefan

Universitatea Tehnică de Construcții București
Facultatea de Hidrotehnica

LifeIsHard

The game

Profesor Coordonator
Fudulu Cosmin
Olteanu Gabriela

Student
Ilie Robert Stefan
Tudor Constantin Mihai

Contents

Introducere 4
Motivatia proiectului 4
Obiective propuse 4
Tehnologii si instrumente hardware si software folosite in implementarea proiectului 5
Notniuni teoretice 7
User 7
Main 8
Casino 10
Blackjack 11
Dices 13
Baccarat 14
Ruleta 17
Slots 19
Deck of cards 21
Jobs 23
Study 25
Story 28
Shop 30
Concluzie 30

Introducere

Motivatia proiectului

Acest proiect de programare reprezintă o simulare simplificată a vieții, în care utilizatorul poate lua anumite decizii legate de cum își petrece timpul sau ce activități face. Ideea de bază a fost să creez un fel de mini-joc în care poți merge la școală, să îți iei un job sau să încerci să faci bani prin diferite metode.

Fiecare acțiune are un efect asupra unui sistem de puncte, care reprezintă progresul sau rezultatul alegerilor făcute de jucător. Proiectul are scop mai mult educativ și a fost o ocazie bună să exersam conceptele de bază în programare, cum ar fi lucrul cu obiecte, cat si sa ne marim cunostintele fata de c++.

Nu este foarte complex, dar am încercat să fie cât mai clar și să funcționeze bine. A fost interesant să combinam mai multe idei într-un singur program și cred că am învățat multe lucruri utile în timp ce l-am făcut.

Obiective propuse

În cadrul acestui proiect, ne-am propus să realizăm o simulare simplificată a vieții, în care jucătorul să poată lua decizii care influențează evoluția personajului său. Am vrut să combinăm mai multe idei – educație, muncă, jocuri de noroc – într-un sistem care să fie și distractiv, dar și logic, cu un scop final.

Obiective principale:

1. Progresie educațională și în funcție de vârstă
   Am creat un sistem prin care jucătorul pornește de la o vârstă fragedă, începând cu grădinița, apoi școala generală, liceul, și în final facultatea. Pe parcurs, vârsta personajului crește automat, iar în funcție de alegeri, anumite opțiuni devin disponibile sau se blochează.
   La finalul facultății, vom avea și un examen, iar rezultatul influențează dacă personajul poate accesa anumite joburi bine plătite sau nu.

2. Alegeri între muncă și învățare – cu impact diferit pe termen scurt și lung
   La anumite vârste, jucătorul va putea alege dacă vrea să continue să învețe sau să înceapă să lucreze.

   Munca aduce bani mai repede, dar uneori limitează progresul educațional.

   Învățatul poate părea mai lent, dar deschide accesul la joburi mai bune sau la mecanici speciale, precum stagii sau cercetare.

Ne-am propus să le balansăm cât mai bine, astfel încât niciuna dintre alegeri să nu fie „cea mai bună” în toate cazurile.

3. Zona de casino – cu patru jocuri de noroc implementate
   Pentru a aduce varietate și o alternativă mai riscantă de a obține puncte/bani, vom introduce o secțiune specială de tip casino, care va conține următoarele jocuri:Baccarat,Blackjack.Ruletă.Sloturi si Dices
   Fiecare joc va avea reguli proprii și șanse de câștig diferite, tocmai pentru a încuraja explorarea. Jucătorul va putea alege dacă riscă banii sau preferă o cale mai sigură.

4. Progresie și diversitate între etape
   Am dorit ca fiecare etapă a vieții să aducă mecanici diferite. De exemplu:

   1. La liceu, jucătorul poate participa la activități extracurriculare.
   2. În facultate, poate alege între mai multe specializări.
   3. La muncă, poate primi promovări sau poate schimba cariera.
      Astfel, jocul nu devine repetitiv și oferă mereu decizii noi, potrivite vârstei și contextului.
   4. Sistem final de puncte / scor La sfârșitul jocului, vom calcula un scor general în funcție de: bani câștigați, nivel de educație, experiență acumulată și alegeri făcute. Acest scor va arăta cât de „reusită” a fost viața simulată de jucător.

Tehnologii si instrumente hardware si software folosite in implementarea proiectului

Tehnologii și instrumente hardware și software folosite

Pentru realizarea acestui proiect, **am folosit o combinație de tehnologii software și hardware** care ne-au ajutat să scriem, testăm și rulăm codul într-un mod organizat și eficient. Alegerea acestor unelte a fost făcută astfel încât să ne putem concentra pe logică și funcționalitate, fără să pierdem timp cu configurări complicate.

Limbaj de programare:

C++
Am ales C++ pentru că este un limbaj puternic, care ne oferă control asupra programului și alocării memoriei. Ne-a permis să organizăm bine codul în funcții și să folosim concepte de bază precum condiții, bucle, funcții și structuri.

Mediu de dezvoltare:

CLion
CLion este un IDE oferit de JetBrains, special conceput pentru C și C++. L-am folosit pentru că oferă autocompletare, evidențiere a erorilor, suport pentru CMake și un debugger integrat. Aceste funcții ne-au ajutat să scriem mai rapid codul și să identificăm mai ușor greșelile.

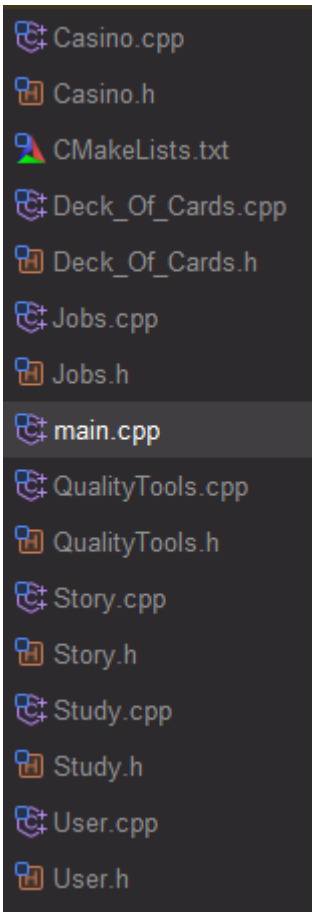
Sistem de operare:

Linux
Proiectul a fost dezvoltat și rulat într-un mediu Linux, pentru că acest sistem de operare este stabil, gratuit și foarte potrivit pentru dezvoltare software. În plus, terminalul Linux ne-a ajutat să rulăm și să testăm aplicația direct, folosind comenzi simple.

Hardware:

Laptopuri personale
Am lucrat la proiect folosind laptopuri personale, fără cerințe hardware speciale. Proiectul nu necesită resurse mari, fiind scris în mod simplu, în linie de comandă.

Notniuni teoretice



Codul a fost creat cu gande de segmentare a tuturor activitatiilor, ele avand obiecte specifice pentru orice.

De exemplu avem main-ul unde programul are main game loop ul si unde este o initializare a utilizatorului.

Fiecare obiect are scop de organizare si de a invata Programarea Orientata pe Obiecte (POO).

Primul obiect prezentat si cel mai important este cel de user.

```
class User {
public:
        int age,money,moneyPerYear,stress, lifeExpentence, health,inteligence, kids, sanity;
        string name,ocupation,study,uniSpec;
        vector<string> degree;
        vector<string> unlockedActivities;
        bool alive, married;

        //////methods///////
        void ShowPlayerStats();
        void KillYourself(bool alive);
        int calculateLifeExpentence();

        bool checkIfYouWannaKys();
};
```

User

Userul are diferite statistici cum ar fii nume, varsta, bani, cati bani face pe an, stresul, cat traieste etc. Dar se specifica si metodele pe care le are clasa, una in care poti sa iti vezi statisticile, poti sa te opresti din joc fortat etc.

Pe partea de metode sunt exemplificate si asezate statisticile userului.

```
void User::ShowPlayerStats() {
    cout<<name<<endl;
    cout<<"Your age is: "<<age<<endl;
    cout<<"Your balance: "<<money<<"RON"<<endl;
    cout<<"You make per month: "<<moneyPerYear<<"RON"<<endl;
    cout<<"Your stress level: "<<stress<<endl;
    cout<<"Your ocupation is: "<<ocupation<<endl;
    cout<<"You study at: "<<study<<endl;
    cout<<"Your inteligence is: "<<inteligence<<endl;
```

Main

```
void start() {
    cout<<"You just were born! Congrats welcome to the real world little one!\nSoon you will understand the real struggles of the
    cout<<"First please enter your name:\n";
    getline(cin, user.name);
    user.moneyPerYear= 0;
    user.money = 10000;
    user.age = 0;
    user.alive = true;
    user.ocupation="";
    user.study="";
    user.uniSpec="";
    user.inteligence=0;
    user.unlockedActivities.push_back("Kind");   user.unlockedActivities.push_back("Wait");   user.unlockedActivities.push_back("
    user.kids = 0;
    user.married = false;
    user.stress = 0;
    user.sanity = 100;

    //user.health = 100;

    //user.lifeExpentence = user.calculateLifeExpentence()+10000;
    cout<<"Hello \n";
    user.ShowPlayerStats();
}
```

In main statisticile utilizatorului sunt setate la valoarea intentionata, practic jocul incepe cand te inasti si termini cand nu mai poti traii.

```
int main() {
    start();

    while (user.alive) {
        story.Activity();


        //user.lifeExpentence = user.calculateLifeExpentence();
        if (user.age >= 60) {
            user.alive = false;
            cout<<"You turned 60! You are too old to live.\n";
        }
    }

    cout<<"Congrats you are dead!";


    return 0;
}
```

Aici are loc bucla principala a jocului unde testeaza daca jucatorul este in viata dupa fiecare an care trece. In caz contrar jocul se va sfarsii.

Casino

În secțiunea de casino, am inclus mai multe jocuri prin care jucătorul își poate testa norocul și abilitățile.
Casinoul funcționează pe baza unei balanțe proprii, din care se retrage o sumă la începutul fiecărui joc. În cazul unui câștig, suma este returnată cu un bonus, în funcție de reguli.

Toate jocurile au fost implementate într-un mod simplu, direct în consolă, fără elemente grafice, pentru a păstra proiectul clar și ușor de testat.

```
class Casino{
public:
    int balance=1000;
    ///////methods
    void ShowGames();
    void Deposit();
    void Withdraw();

    //Casino games
    void Blackjack();
    void Dices();
    void SlotsStart();
    void Roulet();
    void Baccarat();

    void bbnoSS(int balance);
    void playAGame(int game);
};
```

Blackjack

```
How much you want to bet?
Balance: 1000
100
dealer shows: 2
Your hand: 10
Chose what to do:
1.Hit
2.Stay
1
dealer shows: 2
Your hand: 15
Chose what to do:
1.Hit
2.Stay
1
dealer shows: 2
Your hand: 25
You LOST
Your balance is: 900 RON
Do you want to play again? (Type the option number)
1. Play again
2. Exit
```

```
while (choice) {
    if (choice == 1) {
        deck.Shuffle();
        hand=0;
        dealer=0;
        cout<<"How much you want to bet?\n";
        cout<<"Balance: "<<balance<<endl;
        while (true) {
            cin>>pot;
            if (qualityTools.isStringAnInt(pot)) {
                if (stoi(pot) > balance)
                    cout<<"You don't have that much! Try again\n";
                else if (stoi(pot) > 0)
                    break;
                else
                    cout<<"The bet is invalid, try again!\n";
            }
            else
                cout<<"The bet is invalid, try again!\n";
        }
```

```
aux=TransformCardBJ(deck.deck[0].second,0);
deck.deck.erase(deck.deck.begin());
dealer +=aux;
aux=0;
end=false;
while (!end) {
    aux=TransformCardBJ(deck.deck[0].second,0);
    deck.deck.erase(deck.deck.begin());
    hand+=aux;
    cout<<"dealer shows: "<<dealer<<endl;
    cout<<"Your hand: "<<hand<<"\n";
    if (hand>21) {
        end=true;
        cout<<"You LOST\n";
        balance -=stoi(pot);
        break;
    }else {
        cout<<"Chose what to do:\n";
        cout<<"1.Hit\n";
        cout<<"2.Stay\n";
        cin>>choice;
```

```
cout<<"Chose what to do:\n";
cout<<"1.Hit\n";
cout<<"2.Stay\n";
cin>>choice;
if (choice==2) {
    end=true;
    while (dealer<17) {
        aux=TransformCardBJ(deck.deck[0].second,0);
        deck.deck.erase(deck.deck.begin());
        dealer+=aux;
        cout<<"Dealer hit: "<<dealer<<endl;
    }

    cout<<"Your hand: "<<hand<<"\n";
    cout<<"Dealer hand: "<<dealer<<"\n";
    if (dealer>hand&&dealer<=21) {
        cout<<"You lost!\n";
        balance-=stoi(pot);
    }else if (dealer==hand) {
        cout<<"Draw\n";
    }else {
        cout<<"YOU WIN!!!"<<"+ "<<stoi(pot)*2 <<"\n";
```

In cadrul secțiunii de casino a proiectului, am implementat jocul Blackjack, unul dintre cele mai populare jocuri de noroc din lume. Am ales acest joc pentru că este ușor de înțeles, dar totodată implică și puțină strategie, nu doar noroc.

Regulile de bază

Scopul jocului este să obținem o sumă de puncte cât mai apropiată de 21, fără să o depășim. Atât jucătorul, cât și dealerul (calculatorul) primesc cărți și pot decide dacă mai vor cărți sau se opresc.

Valoarea cărților:

Cărțile de la 2 la 10 au valoarea lor numerică.

Valeții (J), damele (Q) și regii (K) valorează 10 puncte.

Asul (A) poate valora 1 sau 11 puncte, în funcție de ce este mai avantajos pentru jucător.

Mecanica jocului în proiect:

1. La început, jucătorul primește două cărți.
2. Se afișează suma punctelor și jucătorul decide dacă:

   „Hit” – mai vrea o carte

   „Stand” – se oprește
3. Dacă trece de 21, pierde automat (bust).
4. După ce jucătorul se oprește, este rândul dealerului, care trage cărți până ajunge la minim 17.
5. La final, se compară scorurile:

   Cine are scor mai mare, dar fără să depășească 21, câștigă.

   Dacă e egalitate, este remiză.

   Dacă dealerul are peste 21, jucătorul câștigă automat.

Blackjacul este implementat prin creerea a doua variabile una pentru jucator alta pentru dealer.

Iar din deck ul de carti este extrasa automat 2 carti pentru utilizator la care le vede suma, iar 2 carti la dealer, unde a doua carte este ascunsa. Urmand astfel, alegerea juicatorului de a sta sau a mai primii o carte.

Dices

```
    }
    else
        cout<<"The bet is invalid, try again!\n";
}
balance -= stoi(bet);

cout<<"On what do you want to bet?\n"
    <<"1. PLayer\n"
    <<"2. Banker\n"
    <<"3. Tie\n";

while (true) {
    cin>>betOn;
    if (qualityTools.isStringAnInt(betOn)) {
        if (stoi(betOn) > 0 && stoi(betOn) < 4)
            break;
        else
            cout<<"The bet is invalid, try again!\n";
    }
    else
        cout<<"Invalid option, try again!\n";
}
deck.Shuffle();

for (int i = 0; i < deck.deck.size(); i++)
```

Jocul Dices este unul dintre cele mai simple jocuri din casino-ul nostru. Ne-am propus să îl facem rapid și ușor de înțeles, bazat doar pe noroc.

Regulile jocului

Atât jucătorul, cât și dealerul (calculatorul) dau câte un zar (valoare între 1 și 6).

Cine are zarul mai mare, câștigă.

Dacă amândoi dau aceeași valoare, dar este 1 și 1, atunci este remiză și se aruncă din nou până când cineva câștigă.În orice alt caz de egalitate (ex: 3 și 3), este considerată remiză fără câștig/pierdere.

```
What do you want to play?
1. Blackjack
2. Dices
3. Slots
4. Roulet
5. Baccarat
2
Game rules:
1. Mostly if you roll higher than the dealer you win and double the sum of your bet
2. If you roll 1 and 1 your win is double than normal
3. If you roll the same thing as the dealer you are forced to roll again, only if you don't roll 1 and 1, in that case you don't lose anything
Do you wish to continue? Type yes or no
yes
How much do you want to bet?
1000
Type anything to start the game
da
You rolled: 6 4
Dealer rolled: 3 3
Congrats you won: 2000
Type anything to continue!
```

```
dealerDices.second == 1)) {
erDices.second) {
ndl;
```

```
void Casino::Baccarat() {


    int choice = 1;
    string bet, betOn;
    vector<int> player, banker;
    int playerTotal, bankerTotal;
    Deck_Of_Cards deck;
    while (choice == 1) {
        int win = 0;
        playerTotal = 0;
        bankerTotal = 0;
        player.clear();
        banker.clear();

        cout<<"How much do you want to bet?\n";
        cout<<"Balance: "<<balance<<" RON\n";
        while (true) {
            cin>>bet;
            if (qualityTools.isStringAnInt(bet)) {
                if (stoi(bet) > balance)
                    cout<<"You don't have that much! Try again\n";
                else if (stoi(bet) > 0)
                    break;
                else
                    cout<<"The bet is invalid, try again!\n";
            }
```

```
void Casino::Dices() {
    int choice = 1;
    pair<int,int> dealerDices, playerDices;
    srand(time(0));
    string answer,n,bet;

    cout<<"Game rules: \n"
        <<"1. Mostly if you roll higher than the dealer
        <<"2. If you roll 1 and 1 your win is double tha
        <<"3. If you roll the same thing as the dealer y
        <<"Do you wish to continue? Type yes or no \n";

    while (true) {
        cin>>answer;
        if (answer == "yes")
            break;
        else if (answer == "no") {
            choice = 2;
            break;
        }
        else
            cout<<"Please enter a valid option\n";
    }
```

Baccarat

```
for (int i = 0; i < deck.deck.size(); i++)
    if (deck.deck[i].second> 10)
        deck.deck[i].second = 10;

player.push_back(deck.deck[0].second);
deck.deck.erase(deck.deck.begin());
banker.push_back(deck.deck[0].second);
deck.deck.erase(deck.deck.begin());
playerTotal = accumulate(player.begin(), player.end(), 0);
bankerTotal = accumulate(banker.begin(), banker.end(), 0);

cout<<"Player's hand: "<<player[0]<<"       "<<"Banker's hand: "<<banker[0]<<endl;

cout<<"Player's total: "<<playerTotal%10<<"       "<<"Banker's total: "<<bankerTotal%10<<endl;

sleep(1);

player.push_back(deck.deck[0].second);
deck.deck.erase(deck.deck.begin());
banker.push_back(deck.deck[0].second);
deck.deck.erase(deck.deck.begin());
playerTotal = accumulate(player.begin(), player.end(), 0);
bankerTotal = accumulate(banker.begin(), banker.end(), 0);
```

```
else if (bankerTotal%10 == 4 && (playersThirdCard > 1 && playersThirdCard < 8))
{
    banker.push_back(deck.deck[0].second);
    deck.deck.erase(deck.deck.begin());
    bankerTotal = accumulate(banker.begin(), banker.end(), 0);
    cout<<"Player's hand: "<<player[0]<<" "<< player[1]<<" "<<player[2]<<"       "<<"Banker's hand: "<<ban

    cout<<"Player's total: "<<playerTotal%10<<"       "<<"Banker's total: "<<bankerTotal%10<<endl;
}
else if (bankerTotal%10 == 5 && (playersThirdCard > 3 && playersThirdCard < 7))
{
    banker.push_back(deck.deck[0].second);
    deck.deck.erase(deck.deck.begin());
    bankerTotal = accumulate(banker.begin(), banker.end(), 0);
    cout<<"Player's hand: "<<player[0]<<" "<< player[1]<<" "<<player[2]<<"       "<<"Banker's hand: "<<ban

    cout<<"Player's total: "<<playerTotal%10<<"       "<<"Banker's total: "<<bankerTotal%10<<endl;
}
else if (bankerTotal%10 == 6 && (playersThirdCard == 6 || playersThirdCard == 7))
```

```
        else if (bankerTotal%10 == 6 && (playersThirdCard == 6 || playersThirdCard == 7))
        {
            banker.push_back(deck.deck[0].second);
            deck.deck.erase(deck.deck.begin());
            bankerTotal = accumulate(banker.begin(), banker.end(), 0);
            cout<<"Player's hand: "<<player[0]<<" "<< player[1]<<" "<<player[2]<<"       "<<"Banker's hand: "<<banker[0

            cout<<"Player's total: "<<playerTotal%10<<"       "<<"Banker's total: "<<bankerTotal%10<<endl;
        }

    }
}


sleep(1);

playerTotal = accumulate(player.begin(), player.end(), 0);
bankerTotal = accumulate(banker.begin(), banker.end(), 0);
```

```
//Cine castiga

if (playerTotal%10 == bankerTotal%10){
    cout<<"It's a tie!\n";
    if (stoi(betOn) == 3)
    {
        win = int(stoi(bet) * 9);
        cout<<"You won: "<<win<<" RON\n";
    }
    else
    {
        win = stoi(bet);
        cout<<"You got your money back\n";
    }

}
else if (playerTotal%10 > bankerTotal%10)
{
    cout<<"Player won! \n";
    if (stoi(betOn) == 1)
    {
        win = int(stoi(bet) *2);
        cout<<"You won: "<<win<<" RON\n";
    }
    else
```

```
else
{
    cout<<"Banker won! \n";
    if (stoi(betOn) == 2)
    {
        win = int(stoi(bet) * 1.90);
        cout<<"You won: "<<win<<" RON\n";
    }
    else
    {
        cout<<"You lost!\n";
    }
}

balance += win;

if (balance > 0) {
    cout<<"Your balance is: "<<balance<<" RON \n";
    cout<<"Do you want to play again? (Type the option number)\n"
        <<"1. Play again\n"
        <<"2. Exit\n";
    cin>>choice;
```

Baccarat este un joc de noroc clasic, pe care l-am inclus în casino pentru a oferi varietate și o alternativă interesantă față de celelalte jocuri. Deși în varianta reală are reguli mai complexe, noi l-am simplificat pentru a putea fi jucat ușor din consolă.

Regulile jocului (versiunea noastră simplificată)

În joc participă două părți: Jucătorul și Bancherul (dealerul).

Se extrag câte două cărți pentru fiecare parte.

Valorile cărților sunt:

Cărțile de la 2 la 9 au valoarea lor.

10, J, Q și K valorează 0 puncte.

A (Asul) valorează 1 punct.

Se adună valorile celor două cărți, iar doar ultima cifră a sumei contează (de exemplu, dacă totalul este 15, scorul este 5).

Cine are scorul mai mare, câștigă.

Reguli speciale:

Dacă este egalitate, nu se pierde și nu se câștigă nimic.

Jucătorul poate paria înainte de extragere pe una dintre opțiuni:

Că Jucătorul va câștiga,

Că Bancherul va câștiga,

Sau că va fi egalitate.

Implementarea functioneaza in felul urmator.

Incepe prin a stabili o suma pe care o pariaza utilizatorul, urmand de niste verificari pentru corectitudinea introducerii de la tastatura. Cat si pe cine pariaza.

```
What do you want to play?
1. Blackjack
2. Dices
3. Slots
4. Roulet
5. Baccarat
5
How much do you want to bet?
Balance: 2018 RON
2018
On what do you want to bet?
1. PLayer
2. Banker
3. Tie
1
Player's hand: 2       Banker's hand: 3
Player's total: 2       Banker's total: 3
Player's hand: 2 10       Banker's hand: 3 10
Player's total: 2       Banker's total: 3
Player's hand: 2 10 6       Banker's hand: 3 10
Player's total: 8       Banker's total: 3
Player won!
You won: 4036 RON
Your balance is: 4036 RON
Do you want to play again? (Type the option number)
1. Play again
2. Exit
```

Deckul de carti este impartit la banca si la jucator.Deckul de carti fiind si el un obiect cu un deck de 52 de carti amestecate. Dupa impartirea cartilor la jucator si banca, se afiseaza usor mainiile totale si se determina un castigator.

Ruleta

Am inclus și jocul de Ruletă în secțiunea de casino pentru a oferi o opțiune clasică, ușor de înțeles și complet bazată pe noroc. Am simplificat regulile pentru a fi potrivite cu formatul jocului nostru, care se joacă în consolă.

Regulile jocului (versiunea simplificată)

Ruleta conține numere de la 0 la 36.

Jucătorul poate paria pe:

Un număr exact

Pe culoare: roșu sau negru

Pe par sau impar

După ce se face alegerea, se generează aleatoriu un număr între 0 și 36.

Numerele pare și impare sunt împărțite pe culori:

Câștiguri:

Dacă jucătorul ghicește exact numărul, câștigul este de 35x suma pariată.

Dacă ghicește doar culoarea sau paritatea (par/impar), câștigul este de 2x.

Dacă nu nimerește deloc, suma pariată este pierdută.

Ruleta functioneaza in felul urmator:

Primesc o suma de bani de la utilizator.

Utilizatorul alege pe ce vrea sa parieze.

Ruleta dupa aceea extrage un numar la intamplare, el fiind inmagazinat intr o variabila de tip pair<int,string> unde vede numarul si culoarea respectiva a zonei alese.

Dupa care programul calculeaza daca ai castigat sau nu .

Cu bineinteles o optiune de a juca iar.

```
void Casino::Roulet() {
pair<int,string> draw = RunRoulet();
switch (choices)
{
    case 1:
    {
        cout<<"1.Black for a 2x on the bet\n"
              "2.Red for a 2x on the bet\n"
              "3.Green for a 36x on the bet\n"
              "Chose bellow\n";
        cin>>secChoice;
        RWait();
        if (secChoice==1)
        {
            if (draw.second=="black")
            {
                RWin(draw,pot);
                balance +=pot*2;
            }else
            {
                Rlose(draw,pot);
            }
        }else if (secChoice==2)
        {
            if (draw.second=="red")
            {
```

```
{
    if (draw.second=="black")
    {
        RWin(draw,pot);
        balance +=pot*2;
    }else
    {
        Rlose(draw,pot);
    }
}else if (secChoice==2)
{
    if (draw.second=="red")
    {
        RWin(draw,pot);
        balance += pot*2;
    }else
    {
        Rlose(draw,pot);
    }
}else if (secChoice==3)
{
    if (draw.second=="green")
    {
        RWin(draw,pot);
```

```
}
case 2:
{
    cout<<"Chose a number to bet
    int numChoice;
    cin>>numChoice;
    RWait();
    if (draw.first==numChoice)
    {
        RWin(draw,pot);
        balance=pot*36;
    }else
    {
        Rlose(draw,pot);
    }
    break;
}
case 3:
{
    cout<<"Chose if the number i
```

```
Hello wellcome to the Roulet!!
How much do you want to bet?
Balance: 1000
1000
Great now what do you want to bet on
1.Black or Red or Green
2.A number
3.Odd or Even
Write bellow the choice
1
1.Black for a 2x on the bet
2.Red for a 2x on the bet
3.Green for a 36x on the bet
Chose bellow
2
It is spining
..
....
......
It landed on 11 black
You lost!!!
```

Slots

```
void Casino::SlotsStart()
{
    int choice,win;
    cout<<"How much you wanna bet?\n";
    cout<<"Your balance: \n"<<casino.balance<<endl;
    cin>>choice;
    if (choice)
    {
        casino.balance -=choice;
        win=Slots(choice);
    }
    casino.balance+=win;
}
```

```
int ShowSlots(vector<vector<int>> hand,int bet) {
    int win=0;
    vector<vector<int>> wH={{0,0,0,1,0,2},{1,0,1,1,1,2},{2,0,2,1,2,2},{0,0,1,1,2,2},{2,0,1,1,0,2}};
    cout<<"\n\n\n\n\n\n\nYour slots are:\n";
    for (int i = 0; i < hand.size(); i++) {
        for (int j = 0; j < hand[i].size(); j++) {
            cout<<hand[i][j]<<"  ";
        }
        cout<<"\n";
    }

    for (int i = 0; i < wH.size(); i++) {

            if (hand[wH[i][0]][wH[i][1]]==hand[wH[i][2]][wH[i][3]]&&hand[wH[i][2]][wH[i][3]]==hand[wH[i][4]][wH[i][5]])
                win=hand[wH[i][0]][wH[i][1]]*bet;
            }

    }
    if (win) {
        cout<<"You won "<<win<<endl;
    }else {
        cout<<"You hit nothing "<<endl;
```

```
int Slots(int bet)
{
    vector<vector<int>> hand={{0,0,0},{0,0,0},{0,0,0}};
    int win=0,winnings=0,choice=1;
    do
    {
        hand=RandomHand(hand);

        win=ShowSlots(hand,choice);
        if (win>0){cout<<"You won "<<win<<endl;}else{cout<<"You lost "<<bet<<endl;
        winnings+=win-bet;
        win=0;
        cout<<"0.Exit\n"
            <<"1.Play again\n";
        cout<<"You have made "<<winnings<<endl;
        cout<<"The bet is "<<bet<<endl;
        cout<<"How much you want to bet, 0 to leave";
        if (winnings-choice+casino.balance<0) {
            cout<<"You dont have any money left to bet that"<<endl;
            return winnings;
```

Pentru a completa zona de casino, am inclus și jocul de Sloturi, într-o versiune simplificată, potrivită pentru rularea în consolă. Am ales să folosim o grilă de 3x3 și în locul simbolurilor clasice (cireșe, lămâi etc.), am folosit numere de la 1 la 6, pentru a fi mai ușor de generat și afișat.

Câștiguri:

3 numere identice pe o linie (orizontala sau diagonală) => câștig in functie de numarul care a fost lina si suma pariată.

Dacă nu există nicio linie de 3 egale pariul este pierdut.Jucătorul poate relua jocul oricând, în limita balanței.

Deck of cards

```
class Deck_Of_Cards {
        public:
        vector<string> suits = {"diamond", "club", "heart", "spade"};
        vector<pair<string, int>> deck;




        void Shuffle();
void Deck_Of_Cards::Shuffle() {
    srand( seed: time( timer: NULL));
    int rng = rand()%10000;
    for (const string& suit : suits) {
        for (int rank = 1; rank <= 13; ++rank) {
            deck.push_back( x: pair<string, int>(suit, [&] rank));
        }
    }
    //random_shuffle(deck.begin(),deck.end());

    shuffle( first: deck.begin(), last: deck.end(), g: default_random_engine(rng));
}
```

Deck of cards este construit din 52 de carti de la 1 la 13 de mai multe culori, inima rosie, inima neagra, romb si trefla. Deck of cards este un obiect ajutator mai ales in Casino dat fiind faptul ca este un mod de a avea jocuri generate la intamplare de fiecare data.

El functioneaza in felul urmator, creem tot pachetul intr un dublu for loop, unde prima data I a valoarea culorii iar dupa ia fiecare numar din pachet la rand, astfel se creeaza un data de tipul vector<pair<string,int>> .

Urmand apoi sa fie amestecat pentru a creea o experienta unica la fiecare joc.

Pachetul de carti functioneaza prin a extrage prima carte din pachet urmand ca aceasta sa fie eliminata din pachet. Iar doar cu o simpla comanda Shuffle, pachet de carti se poate reinoii. Fiind astfel una din structurile necesare la partea de cazino.

Jobs

```
void Jobs::Work() {
    string job = user.ocupation;
    cout<<"You are working as a: "<<job<<"\n";
    if(Jobs::salaries.count(job) != 0) {
        while (true) {
            user.age = user.age + 1;
            user.money += Jobs::salaries[job];
            user.stress += Jobs::sterssLevelPerWork[job];
            cout<<"You earned "<< Jobs::salaries[job]<< " RON this year! \n"
                <<"You now have "<<user.money<<" RON'\n"
                <<"Your stress level is "<<user.stress<<endl
                <<"Do you wish to work for another year? Type yes or no \n";
            string answer;
            while (true) {
                cin>>answer;
                if (answer == "yes")
                    break;
                else if (answer == "no") {
                    return;
                }
                else
                    cout<<"Please enter a valid option\n";
            }
```

```
void Jobs::GetAJob() {
    cout<<"You are looking for a job\n"
        <<"You can get these jobs for now:\n";
        int ind=user.degree.size();
        for (auto posibleJobs :string : user.degree) {
        cout<<ind<<". "<<posibleJobs<<endl;
        }
    cout<<"6. Barista";
    cout<<"Type the job number to get it:\n";
    cout<<"0 to Exit";
    int choice;
    cin>>choice;
    switch (choice) {
        case 0: return;break;

        case 1: user.ocupation = uniSpecToJob[user.uniSpec];cout<<"Congrats you are now a "<<user.ocupation<<"!!";break;

        case 6: user.ocupation="Barista";cout<<"Congrats you are now a Barista!!";break;
        default:"Please write a job number or leave!!\n";Jobs::GetAJob();break;
    }

    user.moneyPerYear=salaries[user.ocupation];
    cout<<"And you earn "
        <<salaries[user.ocupation]<<" per year";
}
```

Jobs este obiectul care se ocupa cu partea de munca,in sensul in care de aici iti poti lua un job, precum poti si muncii.

El functioneaza prin a cauta print toate joburile posibile in stadiul tau curent de viata, de exemplu daca termini o facultate de medicina vei ajunge doctor.

Exista vectori cu salarii unde se regasesc salarile anuale la joburi cat si jobul in sine, in int_req, se gaseste nivel de inteligenta de care ai nevoie pentru a aplica la jobul respectiv, nivelul de stres pe care il primeste este in functie de jobul pe care il ai. Iar acest stress(posibil sanity acum) determina nivel sanatatii tale mentale. Jucatorul fiind nevoit sa experimenteze si sa se adopte unui mod in care stresul (cel mai probabil se cheama santy) sa nu depaseasca un anumit nivel. Astfel se va termina jocul.

Study

```
void Study::StudyHard() {

    if (user.study == "Medical") {
        user.inteligence += intGain;
        user.sanity -= 3;
        uni++;
        if (uni >= 4) {
            cout<<"You passed the university\n";
            cout<<"You know work as a medic";
            user.ocupation = "Doctor";
            user.study = "Homeschool";
        }
    }
    else if (user.age <=3) {
        cout<<"You are too young to study try when you are a bit older (4 years old)!!\n";
        return;
    }
    else if (user.study == ""&&kind ==0&&user.age>=studyAt["kindergarden"]) {
            cout<<"Welcome to the kindergarden!\n";
            user.study="Kindergarden";
            kind++;
    }
    else {
            if (user.age >= studyAt["School"] && kind >= 3 && school == 0) {
                cout << "Welcome to the school!\n";
                user.study="School";
                Study::intGain = 2;
            }
```

```
else {
        if (user.age >= studyAt["School"] && kind >= 3 && school == 0) {
            cout << "Welcome to the school!\n";
            user.study="School";
            Study::intGain = 2;
        }
        if (user.age>=studyAt["Highschool"]&& school >=8 && high == 0) {
            user.study="Highschool";
            cout << "Welcome to the highschool!\n";
            Study::intGain = 3;
        }
        if (user.age>=studyAt["Homeschool"]&& high > 4 && Study::intGain != 3) {
            user.study="Homeschool";
            cout << "Congrats you graduted highschool! Now you are able to learn by yourself\n";
            Study::intGain = 4;
        }


        if (user.study=="Kindergarden") {
            kind++;

        }
        if (user.study=="School") {
            school++;

        }
        if (user.study=="Highschool") {
```

```
        }
        if (user.age>=studyAt["Highschool"]&& school >=8 && high == 0) {
            user.study="Highschool";
            cout << "Welcome to the highschool!\n";
            Study::intGain = 3;
        }
        if (user.age>=studyAt["Homeschool"]&& high > 4 && Study::intGain != 3) {
            user.study="Homeschool";
            cout << "Congrats you graduted highschool! Now you are able to learn by yourself\n";
            Study::intGain = 4;
        }


        if (user.study=="Kindergarden") {
            kind++;

        }
        if (user.study=="School") {
            school++;

        }
        if (user.study=="Highschool") {
            high++;
        }
```

Study este obiectul care se ocupa cu partea de manageriat al invataturii, el se ocupa cu cresterea in gradul de invatare pornind de la gradinita ducandu se la facultate.

Study foloseste un principiu simplu el fiind atunci cand utilizatorul doreste sa invete programul vede in ce clasa se afla el si foloseste un an din viata jucatorului pentru a invata, el avansand astfel cu o clasa in plus.

Study are si modul post liceal unde ai de dat un examen pentru a intra in facultatea dorita.Acest examen poate fi redat de utilizator daca doreste la o data mai indepartata.

Insa daca dai examenul, si inteligenta necesara nu este prezenta, intrebarile din examen nu o sa mai aiba sens. Literele for fii amestecate, schimbate, si inlocuite cu caracter aleatorii.

Story

```
void Story::ActivityUpdate() {

    switch (user.age) {
        case 4:
            Posible_Activites[0].first = 0;
            Posible_Activites[1].first = 1; //unlocks studying
            break;
        case 18:
            Posible_Activites[2].first = 1;
            Posible_Activites[3].first = 1;
            Posible_Activites[4].first = 1;
            Posible_Activites[5].first = 1;
            break;
        case 19:
            if (!Story::everBeenToUni) {
                Posible_Activites[7].first = 1;
            }
            break;
        default:
            break;
    }
}
```

```
void Story::Activity() {
    if (!user.alive)return;
    string choice;
    cout<<"\nWhat would you like to do?\n";
    for (int i=0;i<Posible_Activites.size();i++) {
        if (Posible_Activites[i].first == 1) {
            cout<<i<<". "<<Posible_Activites[i].second<<endl;
        }
    }

    while (true) {
        cin>>choice;
        if (Posible_Activites[stoi(choice)].first != 1 || stoi(choice) > Posible_Activites.size()) {
            cout<<"Please enter a valid option!";
        }
        else
            break;

    }

    switch(stoi(choice)) {
        case 0:
            if (user.checkIfYouWannaKys()) {Story::Activity();return;}
            cout<<"You skiped a year!\n";
            user.age++;
            break;
        case 1:
            if (user.checkIfYouWannaKys()) {Story::Activity();return;}
            study.StudyHard();
```

Obiectul Story este managerul de activitati, el face parte din loopul principal al jcului el fiin apelat mereu, dat fiind faptul ca alegi ce faci. El functineaza prin filtrarea activitatiilor posibile aratand utilizatorului care sunt activitatiile posibile la stagiul lui curent de viata.

Shop

```
void Shop::Store() {
    cout<<"Available balance: "<<user.money<<" RON\n";
    cout<<"What would you like to buy?\n"
        <<"0. Go back\n"
        <<"1. Sanity pill (it restores your sanity by 25%, you can't go over 100%) - 840000 RON\n";

    string choice;
    while (true) {
        cin>>choice;
        if (stoi(choice) >= 0 && stoi(choice) <=1) {
            break;
        }
        else
            cout<<"Please enter a valid option\n";
    }

    switch (stoi(choice)) {
        case 0:
            break;
        case 1:
            if (user.money < 840000) {
                cout<<"You don't have enough money! \n";
                break;
            }

            user.sanity += 25;
            if (user.sanity > 100)
```

Shop are rolul de a achizitiona o pastila de sanity, ea fiind foarte scumpa v a trebuii sa ai grija de sanatatea ta si sa strangi bani pt ea.

Functioneaza pe o decizie care o primeste de la tastatura si o verificare de balanta.

Concluzie

În concluzie, proiectul ăsta a fost un fel de „viață în miniatură”, unde am încercat să punem cap la cap educație, muncă și... puțin noroc prin jocurile de casino. Am pornit cu idei simple, dar pe parcurs ne-am dat seama că avem multe lucruri de legat ca să iasă ceva coerent și jucabil.

Am folosit ce știm din C++, am lucrat în CLion, pe Linux, și ne-am dat seama că nu e chiar ușor să îmbini mai multe mecanici într-un singur program. Totuși, credem că am reușit să facem ceva funcțional și chiar distractiv de testat, chiar dacă nu e perfect sau foarte avansat.

Bibliografie

https://en.wikipedia.org/wiki/C%2B%2B

https://www.jetbrains.com/

https://www.linux.org/

https://www.jetbrains.com/clion/

https://www.geeksforgeeks.org/c-plus-plus/